

DATA/CS 172: Python for Data Analysis

CSV files

Prof. Travis Mandel

10/16/2024



UNIVERSITY
of HAWAII®

HILO

Announcements

- Lab 12 due today

Agenda

- CSV data

So far in this class...

- We started working with toy data
 - Files with one number
- We moved up to more realistic formats
 - Files with many sensor readings
- Lately, we started to touch actual data
 - But only in text format (e.g. Brown corpus)
- TODAY: Open the doors to manipulating many more real-world datasets!



Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

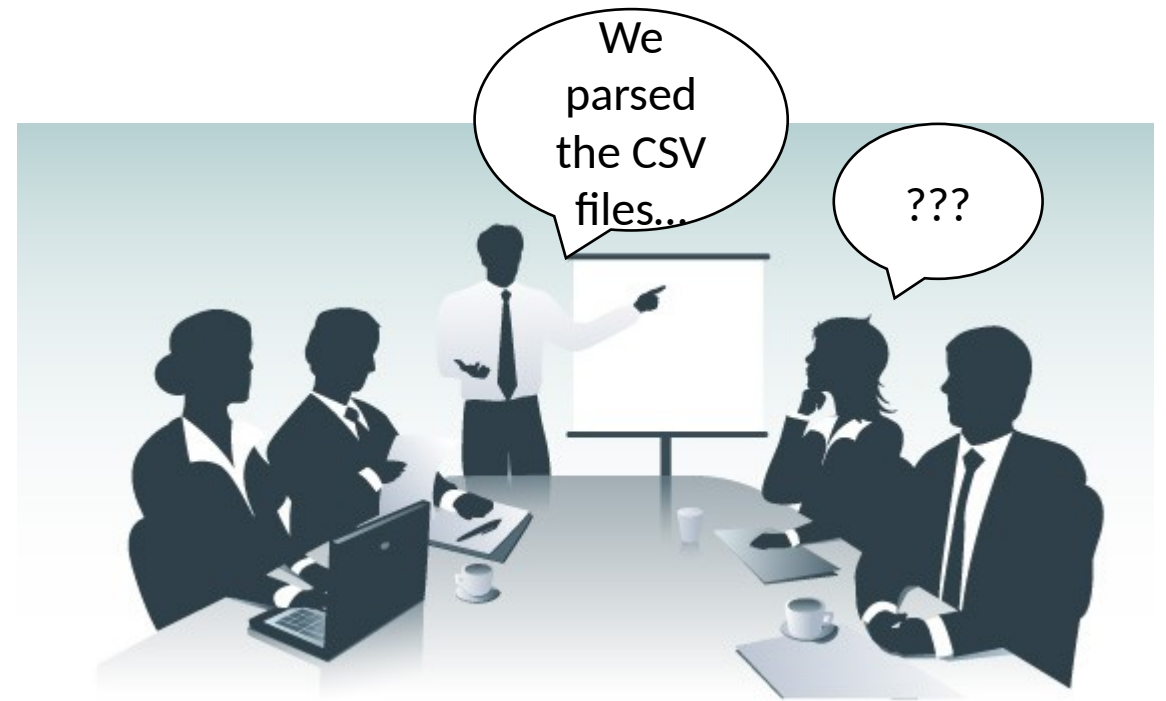
Real World Data



- Is often very complex
- In some projects, custom formats are developed and used (I've been guilty of this)
- In recent years there's been a huge push to standardize formats
 - Helps share data among teams, make better use of existing tools
- Two of the most popular formats are **CSV** and **JSON**
 - Python has special support for these formats
 - We'll cover CSV today, JSON on Friday, APIs Monday

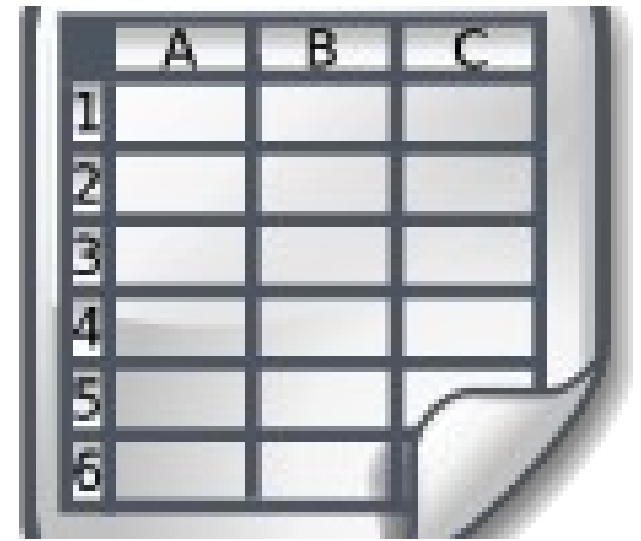
CSV

- Data scientists like to sound really fancy by talking about “parsing CSV files”
 - But CSV just stands for “comma-separated values”
- Basically, CSV files consist of data separated by commas
- Example CSV (demo)



CSV details

- Each line represents a piece of data
- Each piece of data is a list of field values
 - Like columns in a spreadsheet
 - Guaranteed to have the same number of fields in each line
- Fields are separated by comma
 - Often CSV is still used even if the delimiter is changed
- Why use a CSV reader instead of just split?
 - Handles special cases, e.g. what if field has a comma inside it?!
- Although it's not strictly required, most CSV files have a first line which indicates the name of each field. [See Demo]



Parsing CSV files Way #1

```
import csv
f = open("filename.csv", "r")
reader = csv.reader(f)
for row in reader:
    # row is now a list we can manipulate
f.close()
```

- Problems?
 - Keeping track of index is very annoying
 - Have to remember to skip first row

Parsing CSV files Way #2

```
import csv
f = open("filename.csv", "r")
reader = csv.DictReader(f)
for row in reader:
    # row is now a dict with field names as keys
f.close()
```

- **Much better!**
 - More readable, more flexible if column order changes
 - Use this one unless CSV does not have header row

Writing CSV files?

- Writing CSV is less common in data science
- Usually, CSV data comes from real-world data repositories (I like Kaggle)
- However, sometimes the original dataset is too complicated and difficult to work with
 - Writing out a new CSV file can be useful to simplify the format
 - I had to do this for your lab today!



DictWriter

- If we are writing CSV files, we want to include a header row so it is easy to read
 - So a DictWriter is a good choice
 - Works like a DictReader but in reverse
 - Needs to know the fieldnames when created
 - Also needs a special writeheader() call to write the header
 - Need to set lineterminator to avoid blank lines

```
import csv
```

```
f = open("filename.csv", "w")
```

```
writer = csv.DictWriter(f, lineterminator="\n", fieldnames = ["col1", "col2"])
```

```
writer.writeheader()
```

```
writer.writerow({"col1": "row1col1", "col2": "row1col2"})
```

```
writer.writerow({"col1": "row2col1", "col2": "row2col2"})
```

```
f.close()
```

Why are CSV files so widely used?

- Simple format
- Easy to import into spreadsheet application (Google Sheets, Excel)
- Easy to read line-by-line for memory efficiency
- But it has some limitations...

Lab out

- Analyze real-world CSV data to determine if there is any bias in chocolate bar reviews! 😊
- Next time: JSON files!